

A formal verification of the SRP-3 protocol using ProVerif

Aéna ARIA Martin EVRARD Tom FAVEREAU
Liam GRASLIN Aylan LOUNICI

March 13, 2026

Introduction

In this report, we describe our attempt at making a ProVerif model of the SRP-3 protocol in order to prove specific security properties.

We will first make a concise description of the protocol based on its specification, then present our model and the choices we made during its creation, and finally we will give our analysis results and which properties we have proved on our model.

1 Protocol Description

Secure Remote Password (SRP) is a remote authentication protocol which results in a secure key exchange between a client and a host. It follows a challenge-response pattern, allowing the client to prove its identity using a password without revealing it over the network. This protocol does not use any host key or certificate. SRP aims to guarantee a better security against dictionary-based password attacks compared to similar protocols, and to retain its security, even if common low-safety passwords are being used. The version we will verify in this report is SRP-3, as defined in the RFC 2945 [2].

1.1 Message Sequence Chart

We present in figure 1, a message sequence chart for our formalization of the protocol SRP-3. This MSC will use several primitive of our formal modeling. These primitives will be detailed in 2.4.

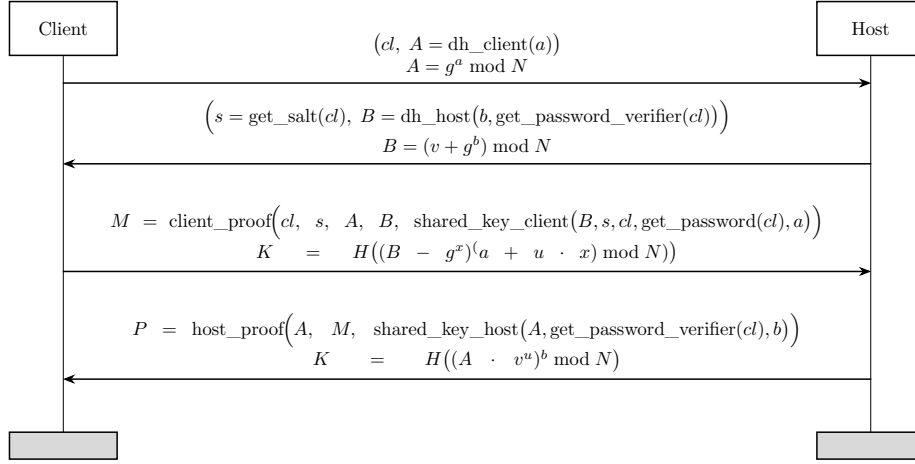


Figure 1: Message Sequence Chart of SRP-3

Each party has to do some integrity checks on the messages to prevent an attackers from deriving the key from the information. The client must abort if the host's public value is $B = 0 \bmod N$. Symmetrically, the host has to abort if the client's sends value $A = 0 \bmod N$ and do so before sending B .

Once both sides has computed the session key, the host verifies the client's proof M by recomputing it from its own K and abort if they do not match. If M is correct, the host returns its own proof that the client verifies and the client abort if it does not match.

1.2 Threat model

We define an attacker as an external actor with complete control of the network: the attacker can read, delay, modify or stop the delivery of messages. Therefore, the attacker can read all of the messages being sent over the network and their content constitute the attacker's knowledge.

There are also a few elements known by all parties at the beginning of a session, including g and N which are defined as constants to be already agreed upon by all participants.

We can also take into account situations where the attacker has gained knowledge from compromised parties: for instance, the password verifier $g^x \bmod N$ or the password itself could have leaked, and it can be interesting to know which security elements are affected in such case. This is especially important, considering that SRP-3 aims to have a good security even in the case of unsafe passwords being used.

1.3 Security goals

We define multiple security goals we would like to ensure.

- **Secrecy of the key:** The key generated by both parties has to be unknown to the attacker.
- **Freshness of the key:** Each key should be new, only generated for a single session.
- **Equality of the key:** The key generated between the client and the host must be the same at the end of the session.
- **Authentication:** If a session has ended with host A (resp. client B) believing to have established a key with client B (resp. host A), we want to verify that a session has been indeed initiated by the expected party.

As explained in the threat model, we can consider each of those properties for different attacker knowledge configurations.

1.4 Cryptographic primitives

The protocol SRP-3 can be summarized using several cryptographic primitives that we aim to describe on a high level. For a better understanding of the protocol we give some details bellow.

The first part of the protocol relies on a Diffie-Hellman like protocol, i.e. modular exponentiation over finite group with base g . Concretely, the client generate a fresh nonce which will serve a secret exponent a and sends $A = g^a \bmod N$ while the server generate a fresh secret exponent b and sends a the value $B = v + g^b \bmod N$ computed from the verifier password v .

Afterwards both the client compute the SRP shared secret S . Using some salt s , a x derived from a hash of the password and the username u .

$$S = (B - k \cdot g^x)^{(a+u \cdot x)} \bmod N \quad (1)$$

And the server computes:

$$S = (A \cdot v^u)^b \bmod N \quad (2)$$

When both parties agree on the protocol parameters these two value should be equal. Hence both parties derived the same session key.

Ultimately both the client and the host process to the authentication step that confirm both parties know the same session key. They are modeled as computation based on some special hash function which is derived from SHA-256. Thought we do not give further details in these notes.

2 Modeling Choices

In order to make a formal verification of this protocol, it is mandatory to go through a modeling process, so that ProVerif may execute queries based on that model. As with all models, it does not (and cannot) capture the entirety of the nuance of the protocol, and this also means that our security proofs are only as good as our model. Therefore, we will explain in this section which modeling choices we made and what is and is not captured by our protocol.

2.1 Protocol modeling

As explained in the protocol description, SRP-3 is based on multiple arithmetic operations, including sums and exponentiations with a modulo. As such, we have decided to **not** represent the logic that stems from the mathematical properties used, as it is beyond the scope of what ProVerif can prove, and adding equations specific to those operations would defeat the purpose of having an automated proof system (as well as needing to prove that our equations are indeed correct, which can be far from trivial in the context of modular arithmetic.)

Therefore, whenever such operation would happen, we transformed it into a simple function that takes as arguments whatever variables are necessary in order to compute the result. For instance, when the host computes $B = (v + g^b) \% N$, we modeled this in ProVerif as `B = dh_host(b, v)` (g and N are omitted as they are fixed constants known of every party). This means that any hypothetical attack using arithmetic properties in order to extract information or to trick a honest party will not be covered in this model, but an attack based on knowledge of the used variables would be represented, as the functions are not defined as private (so the attacker can also use them in our model).

Other than those operations, the modeling of the rest of the protocol is pretty much a one-to-one implementation of the non-formal description, and includes the client-side and host-side checks described in the above sections.

2.2 Threat modeling

ProVerif already gives the attacker control over exchanged messages, and as such we did not have to model the attacker's control over the network. However, to model leaked knowledge from a potential compromised section, we added additional output on a channel reserved to an attacker which can be used in queries with corresponding events. Moreover, the attacker may initiate any number of honest client and host sessions.

2.3 Security properties

Each of the security goals previously enumerated have been modeled using ProVerif queries, including variations depending on what information has leaked.

2.3.1 Authentication and Freshness of the key

Mutual authentication is a security goal that we want to ensure. That is, if a party, let's say the client, authenticates another participant, the host, then the host must have previously started an authentication session with the client on the same data. If the condition is not met, it must mean that credentials have leaked. Moreover, we want to ensure that sessions cannot be replayed using the same data (**Freshness sessions**). Therefore, injective events are required. Both properties (for a client and a host) are modeled using the following queries:

```
1 query cl:id, A:key, B:key, K:key;
2 inj-event(end_host(cl, A, B, K)) ==> inj-event(start_client(cl, A, B, K))
3 || event(password_compromised(cl))
4 || event(db_entry_compromised(cl)).

1 query cl:id, A:key, B:key, K:key;
2 inj-event(end_client(cl, A, B, K)) ==> inj-event(start_host(cl, A, B, K))
3 || event(password_compromised(cl))
4 || event(db_entry_compromised(cl)).
```

The host's identity is not required, as it is not used elsewhere in the model. The agreed data consist only of the Diffie-Hellman public keys and the session key. These are the only values known by both parties and they are specific to the current session.

Note that, these queries also verify the "**Freshness of the key**" property, as the key is part of the data agreed upon during a fresh session.

It is also interesting to know whether these properties are still ensured if one of the two types of compromise occurs. Therefore, we add the following four queries:

```
1 query cl:id, A:key, B:key, K1:key, K2:key;
2 inj-event(end_host(cl, A, B, K1)) && event(password_compromised(cl))
3 ==> inj-event(start_client(cl, A, B, K2))
4 || event(db_entry_compromised(cl)).

1 query cl:id, A:key, B:key, K1:key, K2:key;
2 inj-event(end_host(cl, A, B, K1)) && event(db_entry_compromised(cl))
3 ==> inj-event(start_client(cl, A, B, K2))
4 || event(password_compromised(cl)).

1 query cl:id, A:key, B:key, K1:key, K2:key;
2 inj-event(end_client(cl, A, B, K1)) && event(password_compromised(cl))
3 ==> inj-event(start_host(cl, A, B, K2))
4 || event(db_entry_compromised(cl)).
```

```

1 query cl:id , A:key , B:key , K1:key , K2:key ;
2 inj-event(end_client(cl , A, B, K1)) && event(db_entry_compromised(cl))
3  $\implies$  inj-event(start_host(cl , A, B, K2))
4 || event(password_compromised(cl)).

```

2.3.2 Equality of the key

The protocol should lead to a shared key computed by both parties when mutual authentication succeeds. Therefore, both parties should **compute the same key**. If it is not, it must mean that credentials have leaked. The following query verifies this property:

```

1 query cl:id , A:key , B:key , K1:key , K2:key ;
2 event(end_host(cl ,A,B,K1)) && event(end_host(cl ,A,B,K2))
3  $\implies$  K1 = K2
4 || event(password_compromised(cl))
5 || event(db_entry_compromised(cl)).

```

As before, it is also interesting to know whether this property still holds if one of the two types of compromise occurs. In our implementation, we go further by considering a stronger property¹ : Both calculated keys are the same even if the two types of compromise occurs. The following query models this:

```

1 query cl:id , A:key , B:key , K1:key , K2:key ;
2 event(end_host(cl ,A,B,K1)) && event(end_host(cl ,A,B,K2))
3  $\implies$  K1 = K2

```

2.3.3 Secrecy of the key

Finally, we want the key to **remain secret to the attacker** when mutual authentication succeeds and credentials have not leaked. Therefore, we model this property with the following two queries:

```

1 query cl:id , A:key , B:key , K1:key , K2:key ;
2 event(end_host(cl , A, B, K1)) && event(end_client(cl , A, B, K2))
3 && attacker(K1)  $\implies$  event(password_compromised(cl))
4 || event(db_entry_compromised(cl)).

1 query cl:id , A:key , B:key , K1:key , K2:key ;
2 event(end_host(cl , A, B, K1)) && event(end_client(cl , A, B, K2))
3 && attacker(K2)  $\implies$  event(password_compromised(cl))
4 || event(db_entry_compromised(cl)).

```

¹The choice to verify this "stronger property" was made "a posteriori", after confirming that this property is indeed satisfied (see the Analysis and Results section), and that it subsumes both previous queries.

As before, we also want to determine whether this property still holds if one of the two types of compromise occur. However, as with the previous property, we only verify whether it still holds when both types of compromise occur at the same time with these queries:

```

1 query cl:id , A:key , B:key , K1:key , K2:key ;
2 event(end_host(cl , A, B, K1)) && event(end_client(cl , A, B, K2))
3 && attacker(K1) ==> false .

1 query cl:id , A:key , B:key , K1:key , K2:key ;
2 event(end_host(cl , A, B, K1)) && event(end_client(cl , A, B, K2))
3 && attacker(K2) ==> false .

```

Note that verifying $A \Rightarrow \text{false}$ is equivalent to verifying $\neg A$. Since we were unable to compile the **not** query in ProVerif, we chose to express it using this equivalent formulation.

Moreover, we distinguish the two computed keys in these queries, as their equality is verified by the previous query (we want the queries to be independent from each other). However, in the authentication queries, both keys must be the same to ensure the "Freshness of the key" property.

2.4 Cryptographic abstraction

Our modeling relies on several abstract primitives that aim to capture the main cryptographic core of the protocol. These cryptographic operations are abstracted because our formal verification works symbolically and hence does not relies on such details.

The functions **dh_client** and **dh_host** abstract the Diffie-Hellman phase technically it consists of modular exponentiation over finite group with base g .

The functions **shared_key_client** and **shared_key_host** abstract the computation of the SRP shared secret K . Which is a hash of the modular expression S , computed using some salt s , a x derived from a hash of the password and the username u .

Finally functions **client_proof** and **host_proof** abstract the message authentication step that confirm both parties know the same session key.

3 Analysis Difficulties

3.1 Level of abstraction

The process of modelling a protocol requires a certain level of abstraction regarding certain details of the specification to focus on the core properties of said protocol. The choice of abstraction level will directly influence the relevance and

the accuracy of the analysis. Too coarse, the abstraction may hide vulnerabilities, while too detailed it could lead to an undesirable high complexity. This section discusses the trade-offs made during the modelling.

To ensure a reasonable model, a balance had to be found between realism and abstraction considering cryptographic primitives such as hash functions, the computing of certain intermediate data, as well as some tests. This balance had to be revised several times as the project progressed, and was to us the main, if not the only, difficulty encountered.

Regarding the cryptographic primitives, given that ProVerif operates based on the symbolic model, where said primitives are considered safe by construction, we assumed that, in this case, a high level of abstraction in the model would not be a problem for the later part of the analysis.

The representation of certain computations of intermediate data, that were defined in the protocol specification, such as the results of hash functions, were deliberately omitted to maintain clarity and limit the complexity of our model. The omissions seemed to us to be justified by the focus on more important properties like the mutual authentication of both parties or the session key secrecy.

Certain tests included in the specification, particularly those relating to the values of the public keys of both the client and the host, did not appear to us to have any impact on the protocol's logical security, and seemed to reflect design intentions rather than modelling; they have therefore been omitted.

During the process, it seemed to us that some trade-offs were inevitable, either to achieve realism or to limit the complexity of the model. For this very reason, the initial model was repeatedly refined using ProVerif's specification, examples and manual.

4 Analysis and Results

4.1 Verified properties

In this section, we present the results of our analysis of the protocol SRP-3 using ProVerif. The properties listed earlier (secrecy of the key, freshness of the key, equality of the key and authentication) have been modelled and their soundness was evaluated considering different scenarios, such as a leak of credentials from the client or the host.

4.1.1 Authentication and Freshness

The different queries that have been defined to verify the authentication have allowed us to confirm that: if a client (respectively an host) ends a session thinking it has established a session key with an host (respectively a client), then the latter started a session using the same information. This property

happens to be verified both ways as long as there is no leak of the password or of its verifier.

The defined queries, paired with the use on injective events to prevent an attacker from replaying a session using previous data, helped verify that each computed session key is attached to only one session.

It appears that in the event of a leak, either of the password or of its verifier, the property of authentication is no longer guaranteed; the figure 2 represent the message sequence chart corresponding to the attack rendered possible by the leak of a client password. Gaining the knowledge of this password, an attacker is able to usurp the client identity to the extent of generating a proof that he is in fact mimicked client, upon receiving that proof the host will successfully authenticate the client whom password was leaked.

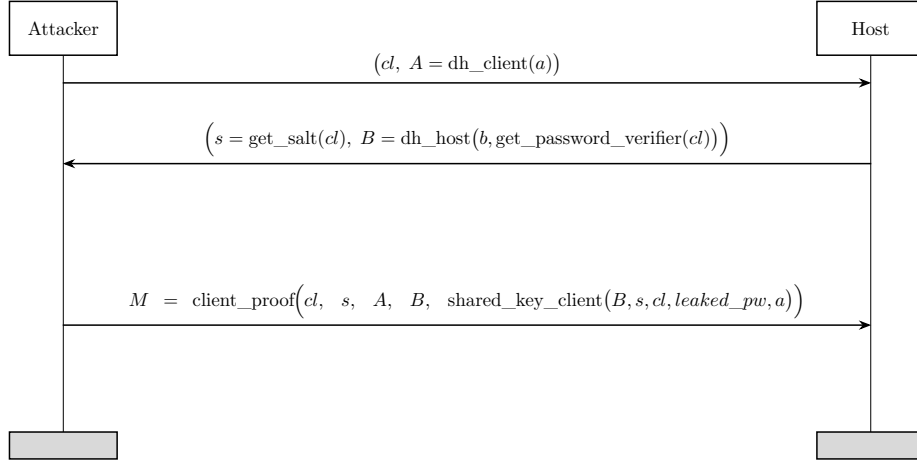


Figure 2: Attack on authentication after a password leak

The figure 3, represent the message sequence chart corresponding to the attacked rendered possible by the leak of the password verifier and the salt associated to a client ID. Gaining the knowledge of these data, an attacker is able to usurp the identity of the host that undergo a data breach in the eventuality of a client trying to establish a session with him, using the newly acquired knowledge, the attacker is able to compute both a valid public key and a valid proof, which once receive by the client will make him authenticate the original host.

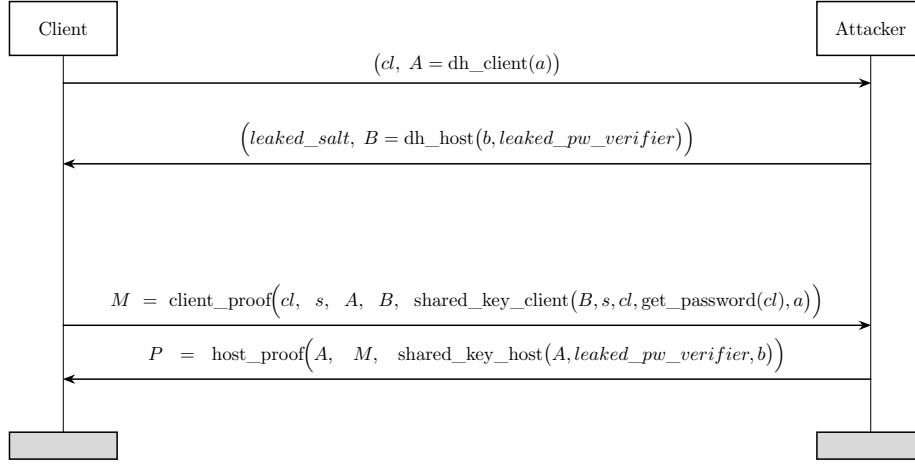


Figure 3: Attack on authentication after a password verifier leak

We can safely assume that this property is ensured outside of our model, as this mostly does not rely on the primitives being used.

4.1.2 Equality of the key

Queries helped verify that the keys computed by the client and the host are the same at the end of a successful session even in the event of a leak of the client's password or of its verifier.

This proof depends on the security of the cryptographic primitives being used, so it is as good as the primitives are.

4.1.3 Session key secrecy

The session key's secrecy property has been verified in order to ensure that an attacker cannot obtain said key. Results show that the key remain secret even in the event of a leak of the password or of its verifier, and such as long as both the client's and the host's secret key remain secret.

This security proof is probably the weakest of all the ones we have given, as the secrecy could potentially be violated in the event of an attacker managing a mathematical trick in order to retrieve informations, either in a passive or active manner, and those attacks are not represented in our model.

4.2 SRP-6 : an improvement of SRP-3

Although the SRP-3 protocol is robust, it has some vulnerabilities, particularly to active attacks. The SRP-6 protocol[1] was introduced specifically to harden

the protocol, as it represents a slight improvement over SRP-3. The main difference was the addition of a k parameter:

$$B = (k \cdot v + g^b) \bmod N \quad (3)$$

where b is a random number that should be at least 256 bits in length and $k = H(N \parallel \text{PAD}(g))$. k prevents an attacker who can select group parameters from being able to launch a 2-for-1 guessing attack. Moreover, we have the parameter u that is a 32-bit unsigned integer which takes its value from the first 32 bits of the SHA1 hash of B . In the new SRP protocol, the value of u has been changed specifically to improve security:

$$u = \text{SHA1}(\text{PAD}(A) \parallel \text{PAD}(B)) \quad (4)$$

This version of SRP is used in a TLS cyphersuite called TLS-SRP.

We did not analyse this protocol, however the vulnerability fixed by this version is mostly a primitive problem, indicating that it is normal to not have found it during our analysis, as we did not consider those arithmetic tricks in our model.

5 Conclusion

This model and analysis, made using ProVerif, have allowed us to verify the different security properties given in the specification of SRP-3; provided, for some, that security assumptions are met (absence of credentials leaks). The results suggest the protocol's strengths, as well as some limits when exposed to data leaks.

References

- [1] David Taylor, Thomas Wu, and Yaron Sheffer. Using the Secure Remote Password (SRP) Protocol for TLS Authentication. RFC 5054, November 2007.
- [2] Thomas Wu. The SRP Authentication and Key Exchange System. RFC 2945, September 2000.